Practical 10

Aim: Optimize Register Allocation Using Graph Coloring: Implement graph coloring to optimize register allocation during code generation. (Use C/C++ language.)

Prac10.cpp:

```
#include <iostream>
#include <string>
using namespace std;

int n = 4;
string var[] = {"a", "b", "c", "d"};
int g[4][4] = {
    {0, 1, 1, 0},
    {1, 0, 1, 1},
    {1, 1, 0, 1},
    {0, 1, 1, 0}
};

void allocate(int k)
{
    int removed[4] = {0}, color[4], stack[4], top = 0;
    cout << "\nRegisters K = " << k << endl;

    for (int c = 0; c < n; c++)
    {
        int pick = -1, maxDeg = -1;
        for (int i = 0; i < n; i++)
        {
            if (removed[i]) continue;
            int deg = 0;
            for (int j = 0; j < n; j++)
                if (!removed[j] && g[i][j]) deg++;
            if (deg < k) { pick = i; maxDeg = -1; break; }
            if (deg > maxDeg) { pick = i; maxDeg = deg; }
        }
        if (maxDeg != -1)
            cout << "Potential spill: " << var[pick] << endl;
        removed[pick] = 1;
        stack[top++] = pick;
    }

    for (int i = 0; i < n; i++) color[i] = -1;
    while (top > 0)
    {
        int v = stack[--top];
        for (int c = 0; c < k && color[v] == -1; c++)
        {
            bool ok = true;
            for (int j = 0; j < n; j++)
                if (g[v][j] && color[j] == c) ok = false;
            if (ok) color[v] = c;
        }
    }

    for (int i = 0; i < n; i++)
    {
        if (color[i] == -1) cout << var[i] << " -> SPILL" << endl;
        else cout << var[i] << " -> R" << color[i] << endl;
```

```
    }
}

int main()
{
    cout << "Interference edges: a-b a-c b-c b-d c-d" << endl;
    allocate(3);
    allocate(2);
    return 0;
}
```

Output:

The interference graph of a, b, c, d is colored with K = 3 and K = 2 registers. Interfering variables must get different registers.

With K = 3 all variables get a register, and a and d share R0 because they do not interfere. With K = 2 the triangle a, b, c cannot be colored, so b is marked as a potential spill and becomes SPILL.